

6 Language Modelling

first four sections: based on slides from Helen Yannakoudakis, Simone Teufel and Ronan Cummins

last section: 'Artificial Intelligence: A modern Approach' from Norvig/Russell 2021

Recap: Ranked retrieval in the vector space model

- Represent the query as a weighted tf-idf vector
- Represent each document as a weighted tf-idf vector
- Compute the cosine similarity between the query vector and each document vector
- Rank document with respect to the query
- Return the top K (e.g., $K = 10$) to the user

Outline

- 1 Query Likelihood
- 2 Estimating Document Models
- 3 Smoothing
- 4 Naive Bayes Classification
- 5 Deep Learning and Neural Networks

Language Model: Query Likelihood

- A model for how humans generate language
- Places a probability distribution over any sequence of words
- By construction, it also provides a model for generating text according to its distribution
- Used in many language-oriented tasks, e.g.,
 - ▶ Machine translation:
 $P(\text{high winds tonight}) > P(\text{large winds tonight})$
 - ▶ Spelling correction:
 $P(\text{about 15 minutes}) > P(\text{about 15 minuets})$
 - ▶ Speech recognition:
 $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

Unigram Language Model

- How do we build probabilities over sequences of terms?

$$P(t_1 t_2 t_3 t_4) = P(t_1)P(t_2|t_1)P(t_3|t_1 t_2)P(t_4|t_1 t_2 t_3)$$

- A **unigram** language model throws away all conditioning context, and estimates each term independently. As a result:

$$P_{uni}(t_1 t_2 t_3 t_4) = P(t_1)P(t_2)P(t_3)P(t_4)$$

What is a document language model?

- A model of how an author generates a document on a particular topic
- The document itself is just one sample from the model (i.e., ask the author to write the document again and he/she will invariably write something similar, but not exactly the same)
- A probabilistic generative model for documents

Two Unigram Document Language Models

Model M_1		Model M_2	
the	0.2	the	0.15
a	0.1	a	0.12
frog	0.01	frog	0.0002
toad	0.01	toad	0.0001
said	0.03	said	0.03
likes	0.02	likes	0.04
that	0.04	that	0.04
dog	0.005	dog	0.01
cat	0.003	cat	0.015
monkey	0.001	monkey	0.002
...	...	...	...

$$\sum_{t \in V} P(t|M_d) = 1$$

Query Likelihood Method I

- Users often pose queries by thinking of words that are likely to be in relevant documents.
- The **query likelihood** approach uses this idea as a principle for ranking documents.
- We construct from each document d in the collection a language model M_d .
- Given a query string q , we rank documents by the likelihood of their document models M_d generating q : $P(q|M_d)$

Query Likelihood Method II

$$P(d|q) = \frac{P(q|d)P(d)}{P(q)}$$

$$P(d|q) \propto P(q|d)P(d)$$

where if we have a uniform prior over $P(d)$ then

$$P(d|q) \propto P(q|d)$$

Note: $P(d)$ is uniform if we have no reason a priori to favour one document over another. Useful priors (based on aspects such as authority, length, novelty, freshness, popularity, click-through rate) could easily be incorporated.

Example I

Model M_1		Model M_2	
the	0.2	the	0.15
a	0.1	a	0.12
frog	0.01	frog	0.0002
toad	0.01	toad	0.0001
said	0.03	said	0.03
likes	0.02	likes	0.04
that	0.04	that	0.04
dog	0.005	dog	0.01
cat	0.003	cat	0.015
monkey	0.001	monkey	0.002
...	...	...	...

$$P(\text{frog said that toad likes frog} | M_1) = (0.01 \times 0.03 \times 0.04 \times 0.01 \times 0.02 \times 0.01)$$

$$P(\text{frog said that toad likes frog} | M_2) = (0.0002 \times 0.03 \times 0.04 \times 0.0001 \times 0.04 \times 0.0002)$$

Example II

Model M_1		Model M_2	
the	0.2	the	0.15
a	0.1	a	0.12
frog	0.01	frog	0.0002
toad	0.01	toad	0.0001
said	0.03	said	0.03
likes	0.02	likes	0.04
that	0.04	that	0.04
dog	0.005	dog	0.01
cat	0.003	cat	0.015
monkey	0.001	monkey	0.002
...	...	...	...

$$P(q|M_1) > P(q|M_2)$$

Estimating Document Models: Documents as samples

- We now know how to rank document models in a theoretically principled manner.
- But how do we estimate the document model for each document?

Example document

click go the shears boys click click click

Maximum likelihood estimate (MLE)

Estimating the probability as the relative frequency of t in d : $\frac{tf_{t,d}}{|d|}$
for the unigram model ($|d|$: length of the document)

Maximum likelihood estimate

click= $\frac{4}{8}$, go= $\frac{1}{8}$, the= $\frac{1}{8}$, shears= $\frac{1}{8}$, boys= $\frac{1}{8}$

Zero probability problem (overfitting)

- But when using maximum likelihood estimates, documents that do not contain all query terms will receive a score of zero.

Maximum likelihood estimates

click=0.5, go=0.125, the=0.125, shears=0.125, boys=0.125

Sample query

$P(\textit{shears boys hair} | M_d) = 0.125 \times 0.125 \times 0 = 0$ (*hair* is an unseen word)

What if the query is long?

Problem in calculation of estimation

- With MLE, only seen terms receive a probability estimate.
- The total probability attributed to the seen terms is 1.
- Remember that the document model is a generative explanation.
- The document itself is just one sample from the model. If a person was to rewrite the document, he/she may include *hair* or indeed some other words.
- The estimated probabilities of seen terms are too big!
- MLE overestimates the probability of seen terms.

Solution: smoothing

Take some portion away from the MLE overestimate, and re-distribute it to the unseen terms.

Solution: smoothing

Discount non-zero probabilities and give some probability mass to unseen words:

Maximum likelihood estimates

click=0.5, go=0.125, the=0.125, shears=0.125, boys=0.125

Some type of smoothing

click=0.4, go=0.1, the=0.1, shears=0.1, boys=0.1, hair=0.01,
man=0.01, their=0.001, bacon= 0.0001, ...

How to smooth I

ML estimates:

$$\hat{P}(t|M_d) = \frac{tf_{t,d}}{|d|}$$

Linear Smoothing:

$$\hat{P}(t|M_d) = \lambda \frac{tf_{t,d}}{|d|} + (1 - \lambda) \hat{P}(t|M_c)$$

M_c is a language model built from the entire document collection.

$\hat{P}(t|M_c) = \frac{cf_t}{|c|}$ is the estimated probability of seeing t in general

(i.e., cf_t is the frequency of t in the entire document collection of $|c|$ tokens).

λ is a smoothing parameter between 0 and 1.

How to smooth II

Linear Smoothing:

$$\hat{P}(t|M_d) = \lambda \frac{tf_{t,d}}{|d|} + (1 - \lambda) \frac{cf_t}{|c|}$$

- High λ : more conjunctive search (i.e., where we retrieve documents containing all query terms).
- Low λ : more disjunctive search (suitable for long queries).
- Correctly setting λ is important to the good performance of the model (collection-specific tuning).
- Note: every document has the same amount of smoothing.

How to smooth III

Linear Smoothing:

$$\hat{P}(t|M_d) = \lambda \frac{tf_{t,d}}{|d|} + (1 - \lambda) \frac{cf_t}{|c|}$$

Dirichlet Smoothing has been found to be more effective in IR where

$$\lambda = \frac{|d|}{\alpha + |d|}$$

~> Dynamic smoothing that changes based on the document length.

How to smooth IV

Plugging this in yields:

$$\hat{P}(t|M_d) = \frac{|d|}{\alpha+|d|} \frac{tf_{t,d}}{|d|} + \frac{\alpha}{\alpha+|d|} \frac{cf_t}{|c|}$$

where α can be interpreted as the background mass (total number of pseudo counts of words introduced).

Bayesian Intuition

We should have more trust (belief) in ML estimates that are derived from longer documents — see the $\frac{|d|}{\alpha+|d|}$ factor.

Putting this all together

Rank documents according to:

$$P(q|d) = \prod_{t \in q} \left(\frac{|d|}{\alpha + |d|} \frac{tf_{t,d}}{|d|} + \frac{\alpha}{\alpha + |d|} \frac{cf_t}{|c|} \right)$$

or

$$\log P(q|d) = \sum_{t \in q} \log \left(\frac{|d|}{\alpha + |d|} \frac{tf_{t,d}}{|d|} + \frac{\alpha}{\alpha + |d|} \frac{cf_t}{|c|} \right)$$

In practice, we use logs - why?

Multiplying lots of small probabilities can result in floating point underflow
[$\log(xy) = \log(x) + \log(y)$].

Pros and Cons

- It is principled, intuitive, simple, and extendable.
- Aspects of tf and idf are incorporated quite naturally.
- It is computationally efficient for large scale corpora.
- More complex language models (markov models) can be adopted and document priors can be added.
- But more complex models usually involve storing more parameters (and doing more computation).
- Both documents and queries are modelled as simple strings of symbols.
- No formal treatment of relevance.
- Therefore model does not handle *relevance feedback* automatically.

Extensions

- **Relevance-based language models** (very much related to Naive-Bayes classification) incorporate the idea of relevance and are useful for capturing feedback
- Treating the query as being drawn from a **query model** (useful for long queries)
- **Markov-chain models** for document modelling
- Use different generative distributions (e.g., replacing the multinomial with neural models)

Classification: Terminology

- **Features:** measurable properties of the data
- Features: the words the text contains, parts of speech, grammatical constructions etc.
- **Classes:** labels associated with the data
 - ▶ Sentiment classification: automatically classify text based on the sentiment it contains (e.g. movie reviews)
 - ▶ Classes: positive or negative sentiment (binary classification)
- Classification is the function that maps input features to a class

Examples of how search engines use classification

- Query classification (types of queries)
- Spelling correction
- Document/webpage classification
- Automatic detection of spam pages (spam vs. non-spam)
- Topic classification (relevant to topic vs. not relevant)
- Language identification (classes: English vs. French etc.)
- User classification (personalised search)

The Naive Bayes classifier I

- The Naive Bayes classifier is a probabilistic classifier.
- We compute the probability of a document d being in a class c as follows:

$$P(c|d) = \frac{P(c)P(d|c)}{P(d)}$$
$$P(c|d) \propto P(c)P(d|c)$$

$P(d)$ is constant during a given classification and won't affect the result.

The Naive Bayes classifier II

$$P(c|d) \propto P(c)P(d|c)$$

$$P(c|d) \propto P(c) \prod_{1 \leq k \leq |d|} P(t_k|c)$$

- $P(t_k|c)$ is the conditional probability of term t_k occurring in a document of class c (**conditional independence assumption**).
- $|d|$ is the length of the document (number of tokens).
- $P(c)$ is the prior probability of c .
- If a document's terms do not provide clear evidence for one class vs. another, we choose the c with highest $P(c)$.

Maximum a posteriori class

- Our goal in Naive Bayes classification is to find the 'best' class
- The best class is the most likely or **maximum a posteriori (MAP) class** C_{map} :

$$C_{map} = \arg \max_{c \in \mathbb{C}} \hat{P}(c|d) = \arg \max_{c \in \mathbb{C}} \hat{P}(c) \prod_{1 \leq k \leq |d|} \hat{P}(t_k|c)$$

Taking the log

- Multiplying lots of small probabilities can result in floating point underflow.
- Since log is a monotonic function, the class with the highest score does not change.
- So what we usually compute in practice is:

$$C_{map} = \arg \max_{c \in \mathbb{C}} \left[\log \hat{P}(c) + \sum_{1 \leq k \leq |d|} \log \hat{P}(t_k | c) \right]$$

Naive Bayes classifier: Interpretation

- Classification rule:

$$C_{map} = \arg \max_{c \in \mathbb{C}} \left[\log \hat{P}(c) + \sum_{1 \leq k \leq |d|} \log \hat{P}(t_k | c) \right]$$

- Simple interpretation:

- ▶ Each conditional parameter $\log \hat{P}(t_k | c)$ is a weight that indicates how good an indicator term t_k is for class c .
- ▶ The prior $\log \hat{P}(c)$ is a weight that indicates how likely we are to see class c .
- ▶ The sum of log prior and term weights is then a measure of how much evidence there is for the document being in the class.
- ▶ We select the class with the most evidence.

Parameter estimation take 1: Maximum likelihood

- Estimate parameters $\hat{P}(c)$ and $\hat{P}(t_k|c)$ from training data - how?
- Prior:

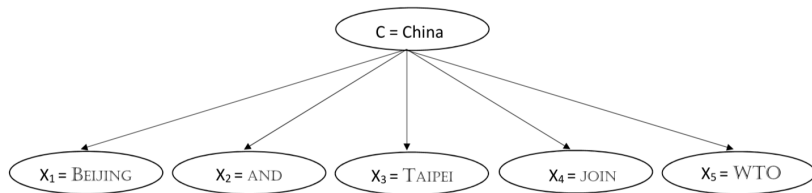
$$\hat{P}(c) = \frac{N_c}{N}$$

- N_c : number of docs in class c ; N : total number of docs
- Conditional probabilities:

$$\hat{P}(t|c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}}$$

- T_{ct} is the number of times t occurs in training documents that belong to class c (includes multiple occurrences).
- We've made a **Naive Bayes independence assumption** here:
 $\hat{P}(t_{k_1}|c) = \hat{P}(t_{k_2}|c)$, independent of positions k_1, k_2 .

The Problem with maximum likelihood estimates: Zeros I



$$P(\text{China}|d) \propto P(\text{China}) \cdot P(\text{BEIJING}|\text{China}) \cdot P(\text{AND}|\text{China}) \\ \cdot P(\text{TAIPEI}|\text{China}) \cdot P(\text{JOIN}|\text{China}) \cdot \textcolor{red}{P(\text{WTO}|\text{China})}$$

- If *WTO* never occurs in class *China* in the training set:

$$\textcolor{red}{\hat{P}(\text{WTO}|\text{China})} = \frac{T_{\text{China}, \text{WTO}}}{\sum_{t' \in V} T_{\text{China}, t'}} = \frac{0}{\sum_{t' \in V} T_{\text{China}, t'}} = 0$$

The Problem with maximum likelihood estimates: Zeros II

- If there are no occurrences of *WTO* in documents in class China ...
- ... we will get $P(\text{China}|d) = 0$ for any document that contains *WTO*!

To avoid zeros: Add-one smoothing

- Before:

$$\hat{P}(t|c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}}$$

- Now: Add one to each count to avoid zeros:

$$\hat{P}(t|c) = \frac{T_{ct}+1}{\sum_{t' \in V} (T_{ct'}+1)} = \frac{T_{ct}+1}{(\sum_{t' \in V} T_{ct'})+|V|}$$

where V is the vocabulary of all distinct words, no matter which class c a term t occurred with.

	docID	words in document	in $c = \textit{China}$?
training set	1	Chinese Beijing Chinese	yes
	2	Chinese Chinese Shanghai	yes
	3	Chinese Macao	yes
	4	Tokyo Japan Chinese	no
test set	5	Chinese Chinese Chinese Tokyo Japan	?

- Estimate parameters of Naive Bayes classifier using the training set.
- Classify test document.

$|text_c| = 8$ (no of tokens in class China)

$|text_{\bar{c}}| = 3$ (no of tokens in other class)

$|V| = 6$ (vocabulary size)

Example: Parameter estimates

Priors: $\hat{P}(c) = 3/4$ and $\hat{P}(\bar{c}) = 1/4$

Conditional probabilities:

$$\begin{aligned}\hat{P}(\textit{Chinese}|c) &= (5 + 1)/(8 + 6) = 6/14 = 3/7 \\ \hat{P}(\textit{Tokyo}|c) = \hat{P}(\textit{Japan}|c) &= (0 + 1)/(8 + 6) = 1/14\end{aligned}$$

$$\begin{aligned}\hat{P}(\textit{Chinese}|\bar{c}) &= (1 + 1)/(3 + 6) = 2/9 \\ \hat{P}(\textit{Tokyo}|\bar{c}) = \hat{P}(\textit{Japan}|\bar{c}) &= (1 + 1)/(3 + 6) = 2/9\end{aligned}$$

The denominators are $(8 + 6)$ and $(3 + 6)$ because the lengths of $\textit{text}_c$ are 8 and 3, respectively, and because the vocabulary consists of 6 terms.

Example: Classification

$$\begin{aligned}\hat{P}(c|d_5) &\propto 3/4 \cdot (3/7)^3 \cdot 1/14 \cdot 1/14 \approx 0.0003 \\ \hat{P}(\bar{c}|d_5) &\propto 1/4 \cdot (2/9)^3 \cdot 2/9 \cdot 2/9 \approx 0.0001\end{aligned}$$

Thus, the classifier assigns the test document to $c = \textit{China}$. The reason for this classification decision is that the three occurrences of the positive indicator *Chinese* in d_5 outweigh the occurrences of the two negative indicators *Japan* and *Tokyo*.

Naive Bayes is not so naive

- Naive Bayes has won some competitions (e.g., KDD-CUP 97; prediction of most likely donors for a charity)
- More robust to non-relevant features than some more complex learning methods
- More robust to concept drift (changing of definition of class over time) than some more complex learning methods
- Better than methods like Decision Trees when we have **many equally important features**
- A good dependable baseline for text classification (but not the best)
- Optimal if independence assumptions hold (never true for text, but true for some domains)
- Very fast; low storage requirements

Time complexity of Naive Bayes

mode	time complexity
training	$\Theta(\mathbb{D} L_{ave} + \mathbb{C} V)$
testing	$\Theta(L_a + \mathbb{C} M_a) = \Theta(\mathbb{C} M_a)$

- L_{ave} : average length of a training doc; L_a : length of the test doc; M_a : number of distinct terms in the test doc; $\mathbb{D}$: training set; V : vocabulary; $\mathbb{C}$: set of classes
- $\Theta(|\mathbb{D}|L_{ave})$ is the time it takes to compute all counts. Note that $|\mathbb{D}|L_{ave}$ is T , the size of our collection.
- $\Theta(|\mathbb{C}||V|)$ is the time it takes to compute the conditional probabilities from the counts.
- Generally: $|\mathbb{C}||V| < |D|L_{ave}$
- Test time is also linear (in the length of the test document).
- Thus: **Naive Bayes is linear** in the size of the training set (training) and the test document (testing). This is **optimal**.

Summary

- Query-likelihood as a general principle for ranking documents in an **unsupervised** manner
 - ▶ Treat queries as strings
 - ▶ Rank documents according to their models
- Document language models
 - ▶ Know the difference between the document and the document model
 - ▶ Multinomial distribution is simple but effective
- Smoothing
 - ▶ Reasons for, and importance of, smoothing
 - ▶ Dirichlet (Bayesian) smoothing is very effective
- Classification
 - ▶ Text classification is **supervised** learning
 - ▶ Naive Bayes: simple baseline text classifier